

Zero to Hero Study Handbook: Payment Fraud Risk Scoring

This handbook is a repository-grounded guide to understand the `payment-fraud-risk-scoring` project from fundamentals to practical mastery.

Scope of this handbook: - Static analysis of repository files only. - Real paths, function/class names, CLI flags, and config keys from this repo. - Focus on core runtime paths: training, model artifact management, batch scoring, and FastAPI serving.

Module 1: Foundations & Architecture

1.1 What this project does

This project trains and serves a fraud-risk model for highly imbalanced card-transaction data (fraud is rare). It supports three production-oriented usage modes: - Training and artifact creation: - Notebook path: `fraud_risk_scoring.ipynb` - Script path: `train_model.py` -> `src/training.py` - Batch scoring: - `batch_score.py` reads input CSV and writes scored CSV. - Online scoring API: - `app.py` provides GET `/health`, POST `/score`, POST `/admin/reload-model`.

Primary business goal: rank and flag likely fraud transactions using metrics suitable for imbalance (PR-AUC, recall, precision, F1, top-K review metrics), not plain accuracy.

1.2 Core paradigms and patterns used here

Definitions first: - Functional utility style: - Standalone pure-ish functions transform inputs to outputs (example: `src/scoring.py` functions like `evaluate`, `best_threshold`, `topk_review`, `risk_score`, `risk_band`). - Object-oriented bundling: - `FraudScorer` dataclass encapsulates model + feature schema + threshold for reproducible inference. - Configuration-as-data: - `TrainingConfig` and `TrainingResult` dataclasses in `src/training.py`. - Environment variable based runtime behavior in `app.py` and `batch_score.py`. - Layered architecture: - Data layer (`src/data.py`) -> training layer (`src/training.py`) -> scoring/inference layer (`src/scoring.py`) -> delivery surfaces (`train_model.py`, `batch_score.py`, `app.py`). - Defensive validation: - Input shape and invariants checked early (binary targets, threshold ranges, required columns, null checks, API schema forbids unknown keys, model hash verification).

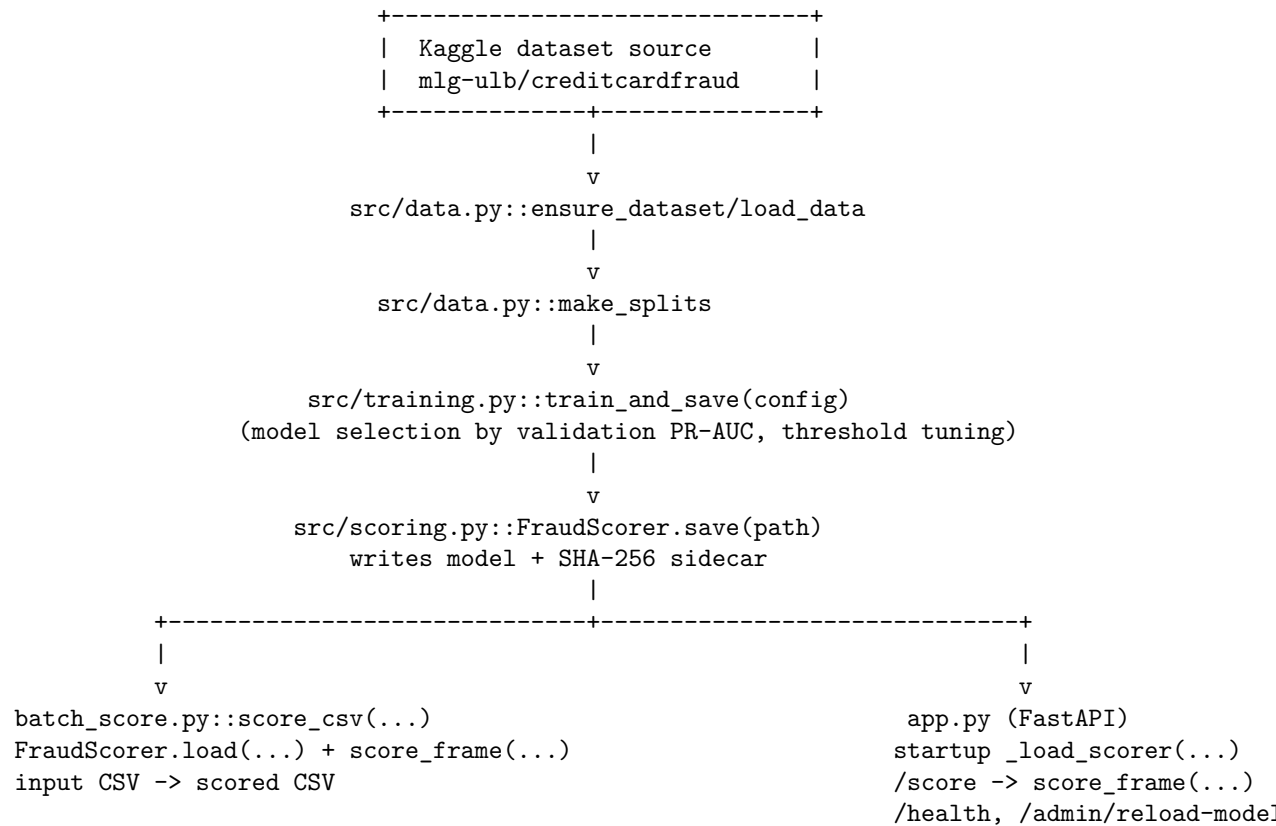
1.3 Architecture overview

Key components: - `src/data.py` - Dataset acquisition (`ensure_dataset`) and splitting (`make_splits`). - `src/training.py` - Candidate model training/selection, threshold tuning, optional SMOTE comparison, artifact saving.

- `src/scoring.py` - Metrics, threshold helpers, risk mapping, `FraudScorer` serialization/hash verification. - `train_model.py` - Script CLI wrapper around `train_and_save`. - `batch_score.py` - CSV scoring CLI around `FraudScorer.load(...).score_frame(...)`. - `app.py` - FastAPI serving, auth, rate limiting, model reload.

Main interaction model: 1. Train once, produce `models/fraud_scorer.joblib` (+ `models/fraud_scorer.joblib.sha256`). 2. Reuse artifact for both batch and API scoring. 3. Keep scoring behavior consistent across surfaces via `FraudScorer`.

1.4 ASCII architecture diagram



Module 2: Repository Map

The table below prioritizes files a new contributor should understand first.

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
README.md	Project overview, quickstart, CLI/API usage contracts, env vars	N/A (documentation)	KAGGLE_API_TOKEN, FRAUD_API_KEY, FRAUD_RATE_LIMIT_PER_MIN, FRAUD_REQUIRE_MODEL_HASH
pyproject.toml	Package metadata and dependencies	N/A	requires-python = ">=3.12.10", dependency list (FastAPI, scikit-learn, XGBoost, LightGBM, etc.)
.python-version	Local Python version pin	N/A	3.12.10
src/data.py	Dataset download/cache, loading, deduplication, stratified splits	ensure_dataset, load_data, feature_columns, make_splits	KAGGLE_DATASET, CSV_NAME, RAW_CSV, TARGET, RANDOM_STATE
src/scoring.py	Evaluation metrics, threshold logic, risk mapping, inference artifact	evaluate, best_threshold, threshold_for_rec, topk_review, risk_score, risk_band, FraudScorer	Hash sidecar path (*.sha256), FraudScorer.threshold, FraudScorer.features
src/training.py	End-to-end training pipeline and model selection	TrainingConfig, TrainingResult, _build_candidate_models, _smote_baseline_for, train_and_save	threshold_beta, try_smote, split models, seed/random state
train_model.py	Script entriypoint for training and saving model/metrics artifacts	parse_args, main	CLI args: --output-model, --metrics-out, --data-csv, --random-state, --test-size, --val-size, --threshold-beta, --no-smote

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
batch_score.py	Batch scoring CLI for CSV input/output	<code>_positive_int</code> , <code>parse_args</code> , <code>score_csv</code> , <code>_render_summary</code> , <code>main</code>	<code>DEFAULT_MODEL</code> , <code>env</code> : <code>FRAUD_REQUIRE_MODEL_HASH</code> , <code>CLI</code> <code>--chunksize</code>
app.py	FastAPI online scoring service	<code>RateLimiter</code> , <code>_configured_model</code> , <code>_load_scorer</code> , <code>_auth_guard</code> , <code>_rate_limit_guard</code> , <code>Transaction</code> , <code>ScoreResponse</code> , <code>HealthResponse</code> , <code>health</code> , <code>reload_model</code> , <code>score</code>	<code>FRAUD_MODEL_PATH</code> , <code>FRAUD_REQUIRE_MODEL_HASH</code> , <code>FRAUD_API_KEY</code> , <code>FRAUD_RATE_LIMIT_PER_MIN</code>
fraud_risk_scoring.ipynb	Not a notebook walkthrough for data analysis, model comparisons, threshold tuning, artifact save demo	Uses <code>src.data/src.scoring</code> helpers; notebook sections 1..11	<code>SEED = 42</code> (in notebook), <code>CHOSEN_THRESHOLD</code> selection in notebook flow
tests/test_scoring.py	Unit tests for scoring logic and hash verification	tests for <code>evaluate</code> , <code>best_threshold</code> , <code>topk_review</code> , <code>FraudScorer.save/load</code>	Edge case expectations: single-class ROC-AUC, hash mismatch failure
tests/test_training.py	Wrapping pipeline artifact tests	<code>training.train_and_validate</code> under mocked model build	Valid model file, hash sidecar, metrics JSON generation
tests/test_batch_score.py	Batch scoring behavior tests	<code>score_csv</code> , <code>_render_summary</code>	Empty-file behavior, output columns, metadata fallback
tests/test_app.py	API-level function tests (without full HTTP server launch)	<code>health</code> , <code>score</code> , <code>_auth_guard</code> , <code>_rate_limit_guard</code>	401 auth behavior, 429 rate-limit behavior, 503 no-model behavior

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
<code>.gitignore</code>	Excludes generated artifacts and secrets from git	N/A	ignores <code>.env*</code> , <code>data/raw/*.csv</code> , <code>models/*.joblib</code> , <code>models/*.sha256</code> , <code>reports/*.csv</code> , <code>reports/*.json</code>
<code>CHANGELOG.md</code>	Versioned change notes	N/A	v0.3.0 additions: hash sidecar, API key gate, reload endpoint, rate limiting

Module 3: Core Execution Flows

3.1 Flow A: Training pipeline (`train_model.py` -> `src/training.py`)

Goal: produce a reusable `FraudScorer` artifact and metrics report.

Step-by-step flow

1. CLI argument parsing:
 - `train_model.py::parse_args()` reads:
 - `--output-model` (default `models/fraud_scorer.joblib`)
 - `--metrics-out` (default `reports/training_metrics.json`)
 - `--data-csv` (optional explicit CSV path)
 - `--random-state`, `--test-size`, `--val-size`, `--threshold-beta`, `--no-smote`
2. Config construction:
 - `train_model.py::main()` builds `TrainingConfig(...)`.
3. Data loading and split:
 - `src/training.py::train_and_save(...)` calls:
 - `dataio.load_data(drop_duplicates=True, csv_path=data_csv)`
 - `dataio.make_splits(df, test_size, val_size, random_state)`
4. Candidate model training:
 - `_build_candidate_models(...)` returns:
 - `LogReg` (scaled logistic regression with class balancing)
 - `RandomForest`
 - `XGBoost` (`eval_metric="aucpr"`, class-weighting by `scale_pos_weight`)
 - `LightGBM`
5. Validation scoring and model selection:
 - For each model, compute validation probabilities and `scoring.evaluate(y_val, p_val, threshold=0.5)`.
 - Select best by max `pr_auc`.

6. Threshold optimization:
 - `scoring.best_threshold(y_val, best_p_val, beta=config.threshold_beta)` (default `beta=2.0`, recall-biased).
7. Optional SMOTE check:
 - If `try_smote=True`, train `_smote_baseline_for(best_name, random_state)`.
 - Keep SMOTE variant only if validation PR-AUC improves over weighted baseline.
8. Final test evaluation:
 - Predict test probabilities.
 - Compute `test_metrics = scoring.evaluate(y_test, test_proba, threshold=threshold)`.
 - Compute top-K review metrics for (50, 100, 200, 500, 1000) via `scoring.topk_review(...)`.
9. Artifact + metrics persistence:
 - Build `FraudScorer(model, features, threshold, metadata)`.
 - Save model via `FraudScorer.save(output_model)` which also writes `.sha256` sidecar.
 - Optionally save summary JSON to `metrics_out`.

Key input and output shapes Training dataset expected columns: - Features: Time, V1 ... V28, Amount (numeric) - Target: Class (binary: 0/1)

TrainingResult fields: - `model_path`: Path - `metrics_path`: Path | None - `best_model_name`: str - `used_smote`: bool - `threshold`: float - `validation`: dict[str, Any] - `test`: dict[str, Any]

`training_metrics.json` (summary) contains keys: - `best_model_name`, `used_smote`, `threshold`, `threshold_fbeta` - `validation`, `test`, `topk_test`, `metadata`, `model_path`

Short real code fragment:

```
best_name = max(results_val, key=lambda model_name: float(results_val[model_name]["pr_auc"]))
threshold, fbeta = scoring.best_threshold(y_val, best_p_val, beta=config.threshold_beta)
scorer_bundle = scoring.FraudScorer(model=best_model, features=features, threshold=float(thr
model_path = scorer_bundle.save(output_model)
```

3.2 Flow B: Batch scoring (batch_score.py)

Goal: score many transactions from CSV and write enriched output CSV.

Step-by-step flow

1. Parse CLI args with `batch_score.py::parse_args()`:
 - required: `--input`, `--output`
 - optional: `--model` (default `models/fraud_scorer.joblib`)
 - optional: `--chunksize` (positive int)

2. Load artifact:
 - `FraudScorer.load(model_path, verify_hash=True, require_hash=_bool_env("FRAUD_REQUIRE_HASH", True))`
3. Score file:
 - `score_csv(input_path, output_path, scorer, chunksize=...)`
 - loops through full DataFrame or chunk iterator
 - each chunk scored via `scorer.score_frame(chunk)`
 - writes CSV header once, appends subsequent chunks
 - tracks (`total_rows`, `total_flagged`)
 - handles truly empty input by writing empty scored schema
4. Render summary log lines:
 - `_render_summary(...)` reports model name, threshold, counts, flag rate, destination.

Key input and output shapes Input CSV: - Must include scorer-required feature columns (same order is not required; names must exist). - If `Class` exists, it is passed through since `score_frame` appends new columns to a copy.

Output CSV appends: - `fraud_probability` (float) - `risk_score` (int, default 0..1000) - `risk_band` ("Low" | "Medium" | "High") - `is_flagged` (0 | 1)

`score_csv(...)` return value: - `tuple[int, int] = (total_rows, total_flagged)`

3.3 Flow C: Online API scoring (app.py)

Goal: serve single-transaction scoring through HTTP.

Startup lifecycle At import/start: - `app.state.model_path = _configured_model_path()`
 - `app.state.scorer = _load_scorer(app.state.model_path)` - `app.state.rate_limiter = RateLimiter(limit_per_window=_int_env("FRAUD_RATE_LIMIT_PER_MIN", 120))`

If model is missing or fails load, app still starts in degraded mode (`scorer=None`).

Endpoint flows

1. GET /health -> `health(request)`
 - returns:
 - `status`: "ok" or "degraded"
 - `model_loaded`: bool
 - `model_path`: str
 - `rate_limit_per_min`: int
2. POST /score -> `score(txn, request)`
 - dependencies:
 - `_auth_guard` (enforces X-API-Key if `FRAUD_API_KEY` is configured)

- `_rate_limit_guard` (per-client fixed window using IP/x-forwarded-for)
 - request model: `Transaction` (`extra="forbid"`, unknown fields rejected)
 - scoring path:
 - single-row `DataFrame` from `txn.model_dump(mode="python")`
 - `scorer.score_frame(row, copy_input=False).iloc[0]`
 - response model `ScoreResponse`:
 - `fraud_probability`: float
 - `risk_score`: int
 - `risk_band`: str
 - `is_flagged`: int
 - `threshold`: float (from loaded scorer)
3. `POST /admin/reload-model -> reload_model(request)`
- dependency: `_auth_guard`
 - reloads scorer from current `app.state.model_path`
 - returns same shape as `/health`

Exact API request shape (`POST /score`) Required JSON keys: - Time, Amount - V1 through V28

All values are float; Amount has `ge=0`.

3.4 Cross-cutting flow: model integrity verification

Where enforced: - `FraudScorer.save(path)` writes `path + ".sha256"` using SHA-256 digest. - `FraudScorer.load(..., verify_hash=True, require_hash=...)`: - if sidecar exists and mismatch -> raises `ValueError` - if sidecar missing and `require_hash=True` -> raises `FileNotFoundError` - if sidecar missing and only `verify_hash=True` -> logs warning and loads

This is used by both `batch_score.py` and `app.py`.

Module 4: Setup & Run Guide

This section explains how to set up the project on a clean machine using only repository-documented tooling (`uv`).

4.1 Prerequisites

- OS: Linux/macOS/Windows with Python support.
- Python: 3.12.10 (`.python-version`, `pyproject.toml` requires-python `>=3.12.10`).
- Package/runtime manager: `uv`.

4.2 Environment setup

```
uv venv --python 3.12.10 .venv
source .venv/bin/activate
```


uv sync

4.3 Dataset and credentials

Data path expected by default: - data/raw/creditcard.csv

If missing, `src/data.py::ensure_dataset()` will attempt Kaggle download through `kagglehub`.

Credential options from repository docs: - `KAGGLE_API_TOKEN` env var, or - `~/.kaggle/kaggle.json`.

4.4 Runtime configuration keys

Key	Used In	Purpose	Default
KAGGLE_API_TOKEN	Dataset download flow (<code>kagglehub</code> auth path)	Enables first-time dataset download	None
FRAUD_MODEL_PATH	<code>app.py::_configure_model_path</code>	On <code>models_path</code> model artifact location	<code>models/fraud_scorer.joblib</code> under app dir
FRAUD_REQUIRE_MODEL_HASH	<code>batch_score.py</code>	Require hash sidecar at model load	<code>true</code> in API/batch defaults
FRAUD_API_KEY	<code>app.py::_auth_guard</code>	Enables API key protection for <code>/score</code> and <code>/admin/reload-model</code>	Not required unless set
FRAUD_RATE_LIMIT_APP	<code>app.py</code> rate limiter	Per-client request cap per minute	120

4.5 Typical command sequences

Train model artifact:

```
uv run python train_model.py \  
  --output-model models/fraud_scorer.joblib \  
  --metrics-out reports/training_metrics.json
```

Batch scoring:

```
uv run python batch_score.py \  
  --input data/raw/creditcard.csv \  
  --output reports/scored.csv
```

Batch scoring with streaming:

```
uv run python batch_score.py \
  --input data/raw/creditcard.csv \
  --output reports/scored.csv \
  --chunksize 50000
```

Run API:

```
uv run uvicorn app:app --reload
```

Notebook path (optional):

```
uv run jupyter lab fraud_risk_scoring.ipynb
```

4.6 Migrations or seeding

There is no database migration or seed system in this repository.

Equivalent initialization steps are: 1. Ensure dataset availability (data/raw/creditcard.csv via local file or Kaggle pull). 2. Train and persist model artifact (models/fraud_scorer.joblib + .sha256). 3. Use artifact in batch/API paths.

Module 5: Study Plan & Practice Exercises

5.1 Ordered study plan for a new learner

1. Read README.md end-to-end to get product intent and runtime surfaces.
2. Read src/scoring.py first to understand core fraud metrics, thresholding, and FraudScorer.
3. Read src/data.py to learn data acquisition and split discipline.
4. Read src/training.py to see model selection policy and artifact metadata composition.
5. Read train_model.py and batch_score.py to understand CLI wrappers and operational behavior.
6. Read app.py to learn request schema validation, auth, rate limiting, and degraded startup strategy.
7. Read tests/ as behavioral specification for edge cases and expected contracts.
8. Skim fraud_risk_scoring.ipynb sections 8-11 to connect tutorial exploration with production code paths.

5.2 Practice exercises (with file-targeted prompts)

1. Exercise: Explain why model selection is based on PR-AUC, not accuracy.
 - Where to read: README.md, src/scoring.py module docstring.
2. Exercise: Trace exactly how the training threshold is chosen and where it is stored.
 - Where to read: src/training.py, src/scoring.py::best_threshold, FraudScorer.

3. Exercise: List every validation that `FraudScorer.score_frame` performs before scoring.
 - Where to read: `src/scoring.py::FraudScorer.score_frame`.
4. Exercise: Describe the API security and abuse controls in order of execution for `POST /score`.
 - Where to read: `app.py` dependencies and guard functions.
5. Exercise: What happens when the model file is missing at API startup vs request time?
 - Where to read: `app.py::_load_scorer`, `_get_scorer`, `health`, tests in `tests/test_app.py`.
6. Exercise: Reconstruct the JSON structure written to `reports/training_metrics.json`.
 - Where to read: `src/training.py` (summary dict in `train_and_save`).
7. Exercise: Explain how empty CSV input is handled in batch scoring.
 - Where to read: `batch_score.py::score_csv`, `tests/test_batch_score.py`.
8. Exercise: Explain exactly when `FraudScorer.load` raises `ValueError` vs `FileNotFoundError`.
 - Where to read: `src/scoring.py::FraudScorer.load`, `tests/test_scoring.py`.
9. Exercise: Enumerate all environment variables and defaults that change runtime behavior.
 - Where to read: `README.md`, `app.py`, `batch_score.py`.
10. Exercise: Identify where train/val/test leakage is reduced in this project.
 - Where to read: `src/data.py::load_data` (dedup), `make_splits` (stratification), training flow order.

5.3 Model answers / solution outlines

1. PR-AUC is robust for extreme class imbalance and reflects ranking quality on positives; accuracy is misleading when negatives dominate.
2. Threshold comes from `scoring.best_threshold(y_val, best_p_val, beta=threshold_beta)` in `train_and_save`, then saved in `FraudScorer.threshold`.
3. `score_frame` validates scorer invariants, DataFrame type, required feature columns, empty-frame branch, null checks, `predict_proba` shape, and output length consistency.
4. `POST /score` applies `_auth_guard` (if API key configured), then `_rate_limit_guard`, then model-readiness check in `_get_scorer`, then scoring.
5. On startup, missing model yields degraded app (`scorer=None`); at `/score`, `_get_scorer` raises HTTP 503 until model is available/reloaded.
6. Metrics JSON includes model choice, SMOTE usage, threshold and F-beta, validation/test metrics dictionaries, top-K metrics list, metadata, and model path.
7. If no rows are yielded, batch flow reads header-only frame, scores empty frame, and still writes output schema with scoring columns.
8. `ValueError` on hash mismatch; `FileNotFoundError` when hash is required but missing.
9. Runtime vars: `KAGGLE_API_TOKEN`, `FRAUD_MODEL_PATH`, `FRAUD_REQUIRE_MODEL_HASH`,

`FRAUD_API_KEY`, `FRAUD_RATE_LIMIT_PER_MIN`.

10. Leakage controls: drop exact duplicate rows before splitting, stratified split before model fitting, deterministic random seed.

Learner Verification Checklist

Use this checklist to confirm end-to-end understanding:

- I can explain the role of each module in `src/` and how they connect.
- I can trace the full training path from dataset load to `FraudScorer.save`.
- I can explain how `best_threshold(..., beta=2.0)` affects fraud operations.
- I can describe the exact request and response schema for `POST /score`.
- I can explain how hash sidecar verification protects model integrity.
- I can explain how batch scoring handles both large files (`--chunksize`) and empty files.
- I can list all runtime environment variables and their defaults.
- I can point to the tests that validate API degraded mode, auth, rate limiting, and scoring edge cases.
- I can describe why this project uses PR-AUC/recall/top-K metrics instead of accuracy.
- I can explain what artifact files are produced by training and how downstream paths consume them.